Laboratory: Fighting Forest Fires with Distributed Messaging

In this lab you will design and implement a decentralised distributed system that coordinates fire trucks fighting forest fires across a 20×20 grid. Along the way you will also observe how the underlying network protocols carry your messages, by using a packet sniffer on your own traffic.

The lab combines two themes from the lectures: **observing real network traffic** (so that the abstractions of distributed systems become tangible) and **building distributed coordination** (so that you experience the design trade-offs first hand).



Figure 1: Fire trucks fighting forest fires.

> Fire trucks must collaborate because they share a global, scarce resource: **the water reservoir at the fire department**, which can refill only a fixed amount of water per timestep, and which can serve only one truck at a time.

There is no central controller. The fire trucks must coordinate solely via asynchronous messaging.

The lab consists of six tasks. **Task 0** familiarises you with the provided template. **Tasks 1–4** build the distributed system in clearly delimited steps, each with its own learning goal. **Task 5** is the presentation of your solution.

The simulation model

You will work with the following simulation rules. They are already implemented for you in the template (`task-0-template.go`).

- A 20×20 grid represents the forest. Cell $(0, 0)$ is the **fire department**; fires cannot ignite there.
- Each timestep, an existing fire may grow in intensity. Every x timesteps, every fire **spreads to its four neighbouring cells**.
- A firetruck moves one cell per timestep (north, south, east or west) or performs one action (extinguish).
- **Extinguishing** a fire of intensity i costs one unit of water per action and removes one intensity level; a fire is extinguished when its intensity reaches zero.
- A firetruck holds at most y units of water on board; water is consumed by extinguishing.
- The fire department has an **unlimited** supply of water but can serve **only one truck at a time**, refilling it at a rate of z units per timestep.

The constants x, y, z are defined at the top of `task-0-template.go`.

Part 0: Familiarise yourself with the template (Estimated duration: ∼2 h)

Goal of Part 0. Understand the simulation model and the structure of the provided code so that you have a confident starting point for Tasks 1–4.

The template we provide implements the whole simulation as a single sequential Go program. A main loop advances the simulation one timestep at a time: fire dynamics are updated, each truck takes one action, and the grid is redrawn in place. The purpose of Task 0 is simply to understand this structure before you start pulling it apart.

Tasks in Part 0

a) Clone the lab template repository and open `task-0-template.go` in your editor from `https://git.chalmers.se/heyn/dit357_lab`.

b) Compile and run the template: `go run task-0-template.go`.

You should see a 20×20 grid updated in place every 0.6 s. Two trucks (`T1`, `T2`) roam the grid, fight fires. Once their water runs out they head back to the fire department at cell $(0, 0)$ and *idle* there.

c) Read the code. Identify:

(a) the three data structures (`World`, `Cell`, `Truck`) and what each field represents;

(b) the main simulation loop and the order in which fire dynamics, truck actions, and the display happen each timestep;

(c) the `act` method which chooses a truck's single action each timestep.

d) Sketch a flow diagram of the `act` method showing the three actions a truck can take and the condition under which each action is taken.

No code changes are required for Task 0.

Part 1: Design an application-layer protocol over TCP (Estimated duration: ∼8 h)

Goal of Task 1. Design an application-layer protocol for water refill requests, document it in the style of the application-layer exercise, and implement both sides in Go.

When the fire trucks in Task 0 run out of water, they return and idle at the station. That is not useful for fire fighting. Your task is to implement a **fire-department server** that handles water refill requests over TCP. The server runs as a **goroutine** alongside the simulation, and the trucks connect to it as clients when they need water.

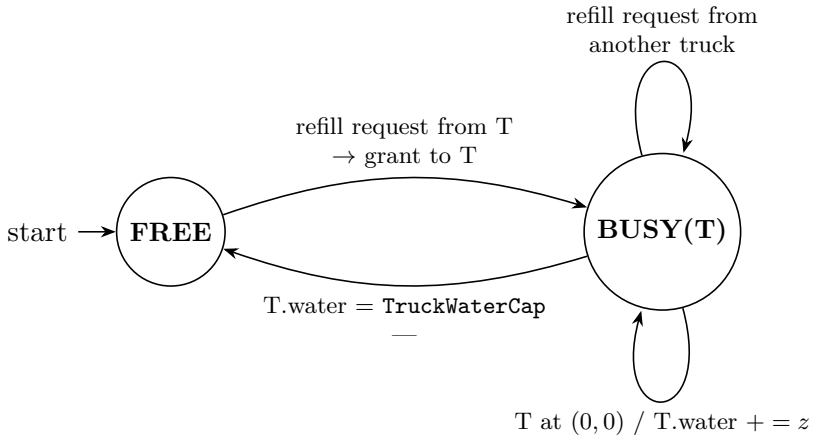
Goroutine in a nutshell. A **goroutine** is a lightweight function that runs concurrently with the rest of your program. You launch one by writing `go f(args)` instead of `f(args)`. Goroutines are far cheaper than OS threads (a Go program may run thousands of them) and are managed by the Go runtime. In this task you will run the fire-department server as a goroutine so that it can accept TCP connections in the background while the simulation loop keeps running in the main goroutine.

What the fire department must do (server side)

Your server manages access to the single water reservoir inside the fire station. Whatever protocol you design, the resulting behaviour must satisfy the following rules:

1. The reservoir can serve only **one truck at a time**. While one truck is being served, the others must wait.
2. A truck that has run out of water requests a refill, travels to cell $(0, 0)$, and is refilled there at a rate of z units per timestep until its tank reaches `TruckWaterCap`.
3. Once a truck is full, the reservoir becomes available for the next truck.

The server can be modelled as the two-state machine below. **FREE** means the reservoir is available; **BUSY(T)** means truck T is currently refilled.



Truck behaviour (client side)

A truck that runs out of water must, each timestep:

1. Request a refill from the server according to your protocol.
2. If access is granted: stop fighting fires and navigate toward $(0, 0)$. The server refills the tank on arrival.
3. If access is not (yet) granted: retry on a later timestep according to your protocol's rules.

Designing the protocol

Specify your protocol exactly as you did for the smart-coffee-machine protocol in the application-layer exercise. You must include:

1. **A message catalogue.** For each direction (truck → server and server → truck), list every message, its syntax, and its purpose. Use a text-based, human-readable encoding. For example, **Newline-delimited JSON** (one JSON object per line, `\n`-terminated) is a possible choice, but the design is yours.
2. **The actions** each side takes on sending or receiving each message.
3. **A normal-operation arrow diagram**: a successful refill, drawn as a simple message-sequence sketch.
4. **A contention arrow diagram**: what happens when a second truck requests water while the reservoir is busy.
5. **Transport assumptions.** State what your protocol assumes of the transport layer, and explain why **TCP** (reliable, in-order, connection-oriented delivery) satisfies those assumptions.

Tasks in Part 1

a) Design your refill protocol and document it (message catalogue, actions, two arrow diagrams, transport assumptions) in your code repository.

b) Create a fire-department server goroutine in `main`. The server listens on `localhost:9000` and implements the FREE/BUSY behaviour based on your protocol.

c) Update the truck's `act` method: when `water == 0`, run your refill protocol as a client, handling both the FREE/Granted and the BUSY/Waiting case.

d) Implement the refilling: each timestep the server checks whether the served truck is at $(0, 0)$; if so, it increments that truck's water by z, up to `TruckWaterCap`, then returns to FREE.

e) Run the simulation and verify that trucks refill and return to firefighting.

Part 2: Observing the protocol with Wireshark (Estimated duration: ∼6 h)

Goal of Task 2. Capture the refill TCP exchange on the loopback interface, identify the message payloads inside TCP segments, and connect what you observe to your knowledge of TCP from the lectures.

In Task 1 you designed and implemented a text-based protocol. In Task 2 you step down one level of abstraction and watch those same messages at the TCP segment level, exactly as you studied in the lectures on the application and transport layers.

Wireshark installation. If you have not yet installed Wireshark on your machine, follow the instructions in the companion document *DIT357_Wireshark.pdf* before starting this task.

Summary of Task 2

a) Start your Task 1 simulation in one terminal.

b) Start Wireshark and select the **loopback interface** (see table below for the name on your platform).

c) Apply the display filter `tcp.port == 9000` (replace `9000` with the port your server uses if you chose a different one).

d) Let the simulation run until at least two complete refill exchanges have occurred. You will see them appear in the packet list.

e) Stop the capture and answer the questions in the "Analysis questions" section below.

Operating system	Loopback interface name
Ubuntu Linux	`lo`
macOS	`lo0`
Windows	`Adapter for loopback traffic capture`

Starting the capture

Start Wireshark. You will see a list of network interfaces on your machine, as shown in Figure 2. Double-click the loopback interface to begin capturing. Active interfaces show a small traffic graph next to their name.

Once capturing has started, type `tcp.port == 9000` in the display-filter bar at the top and press Enter. This hides all traffic not involving your server's port and makes the relevant exchanges easy to spot. Figure 3 shows what the capture screen looks like with packets being recorded. Let the simulation run until you can see at least two request/reply pairs in the packet list, then stop the capture (red square button, or **Capture** → **Stop**).

Analysis questions

Select individual TCP segments in the packet list (middle pane shows decoded protocol fields; bottom pane shows raw bytes) and answer the following questions. Include your answers, with supporting screenshots, in your code repository.

1. **Reading the payload.** Find one complete refill exchange. Select the TCP segment carrying your refill *request*. Can you read the message string in the raw-bytes pane at the bottom? Which TCP segment carries the *grant* response from the fire department?

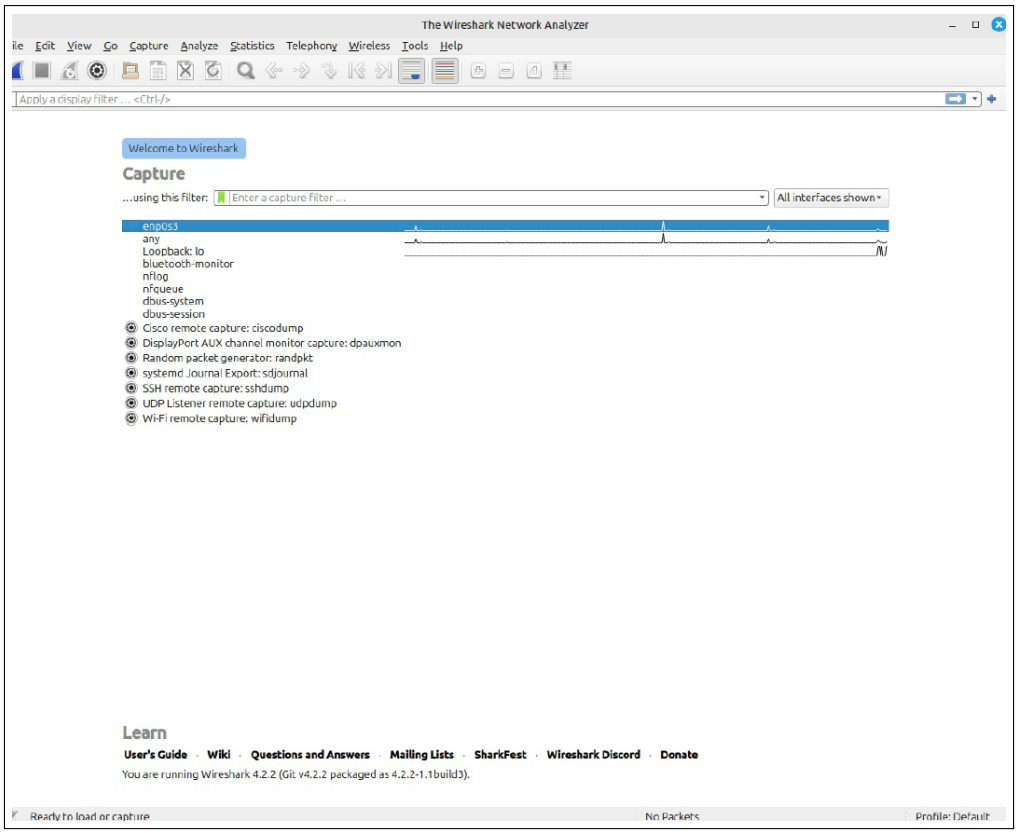


Figure 2: Wireshark welcome screen and interface list. Double-click the loopback interface to start capturing. The interface names shown here are from a Linux system; on macOS the loopback entry is `lo0`, and on Windows it is labelled `Adapter for loopback traffic capture`.

2. **Segment count.** How many TCP segments are exchanged for a single request/reply pair (including the connection setup and teardown)? Which side, truck or server, initiates the connection? Which side sends the first FIN?

3. **Busy station.** Trigger the situation where the server is busy and a second truck attempts to request water at the same time (e.g., by increasing the number of trucks in the field). What does Wireshark show for that request? Does the wire trace match what your protocol design says should happen in the busy case?

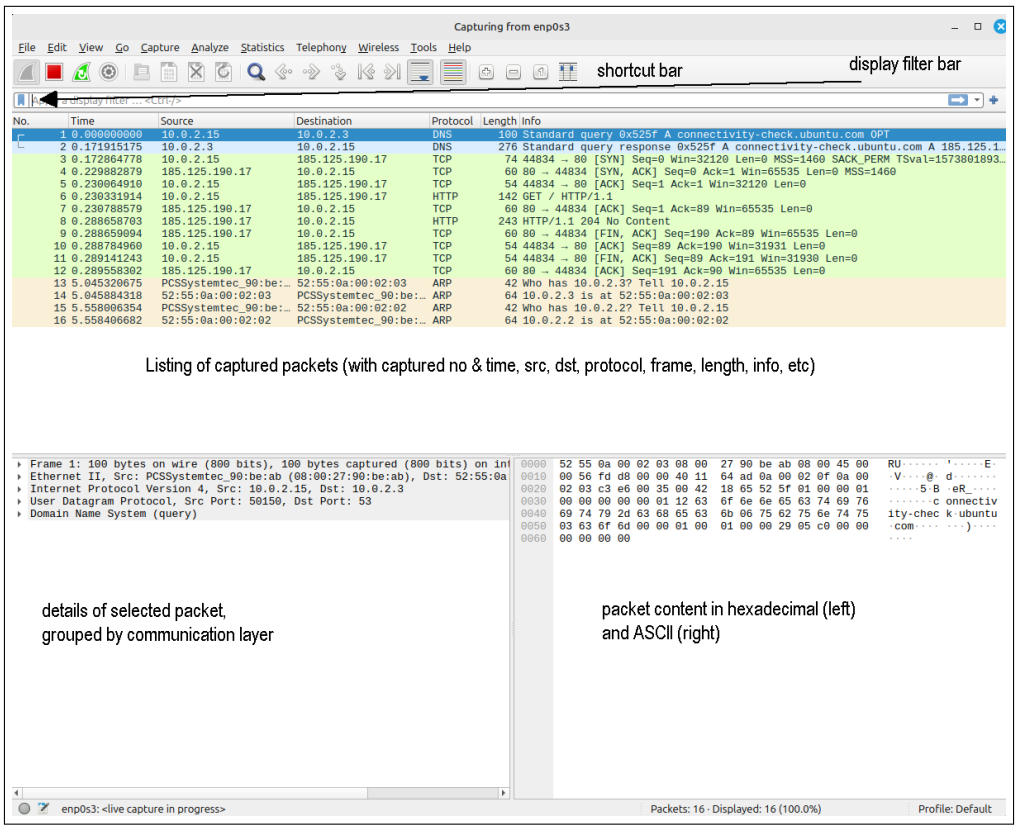


Figure 3: Wireshark capture screen showing packets on the loopback interface. Each row is one TCP segment. Select a row to see the protocol layers in the middle pane and the raw bytes in the bottom pane.

Part 3: NATS messaging, naming and decentralised cooperation (Estimated duration: ∼12 h)

Goal of Part 3. Migrate the refill protocol from raw TCP to a NATS request-reply architecture. Then design a naming scheme suitable for a distributed setting, and implement a decentralised call-for-help protocol using the NATS publish-subscribe architecture that lets trucks request and offer help with no central decision-maker.

So far all your network traffic has been raw TCP that you wired up yourself. In the real-world™, asynchronous distributed systems are typically built on top of a *message broker* that handles addressing, queueing and delivery for you. In Task 3 you first migrate the current refill protocol from raw TCP to a **NATS** request-reply architecture. You then give the trucks a way to **name** each other and the world around them, and a way to **call for help** when a fire is too big to handle alone.

Migrate refills to NATS request–reply

Replace the TCP server from Task 1 with a NATS subject (for example `firedept.refill`) using the request-reply pattern. The fire-department process subscribes to the subject; trucks publish requests and await replies. Run a local NATS server in a separate terminal (`nats-server -p 4222`) and connect to it from your Go processes using the `nats.go` client library.

Discuss in your group. What did NATS take off your hands compared to the raw-TCP version (addressing, connection setup, reconnection, delivery semantics, fan-out)? What new behaviours did it introduce (for example at-most-once delivery, or the absence of any built-in causal ordering)? Where did your Task 1 protocol simplify, and where did it have to change?

Naming and calling for help

In Tasks 1 and 2 the trucks roam blindly and only talk to the fire department. Now you give them identities and a broadcast channel of their own. The help channel shall be decentralised: no central scheduler decides who responds. You build it using the **NATS publish-subscribe** architecture.

Pub/sub in a nutshell. In the publish-subscribe architecture, a sender *publishes* a message to a named subject without knowing who will receive it. Any number of *subscribers* to that subject receive a copy. This decouples senders from receivers and naturally supports one-to-many, exactly what a broadcast "who can help?" call needs. In NATS you publish to a subject (for example `trucks.help`) and every subscribed truck reacts independently.

Discuss in your group. Why is pub/sub a better fit for the call-for-help channel than the request–reply pattern you used for refills? What delivery guarantees does your help protocol have, and what happens to a help call that no truck answers?

Tasks in Part 3

a) **(NATS)** Start a local NATS server. Replace the TCP refill channel from Task 1 with a NATS request-reply exchange on a subject such as `firedept.refill`. Verify that trucks still refill correctly.

b) **Naming.** Define a naming scheme for (i) trucks and (ii) fire locations. State whether it is flat, structured, or attribute-based, and motivate the choice in your report. You will need to address the fact that the set of trucks may grow at runtime (item e).

c) **Big fires.** Define what a "big fire" means in your simulation (for example a fire of maximum intensity, or a cluster of k adjacent burning cells). Implement detection by the truck that observes it.

d) **Call for help (NATS pub/sub).** Implement a **decentralised call-for-help protocol** on a NATS pub/sub subject (for example `trucks.help`). A truck encountering a big fire publishes a help request. All trucks subscribe; each one decides *itself* (no central decision) whether and how to respond, based on a rule of your design (distance, current water level, current task, etc.). Describe the protocol in your repository.

e) **Deployment of additional trucks.** If too many trucks call for help (you decide what too many is here), the fire department deploys additional trucks. Implement a mechanism that listens to the "call-for-help" NATS subject and reacts by deploying more fire trucks if needed.

f) **Fuel saving.** If a truck does not encounter any fire for q consecutive timesteps it returns to the fire department to save fuel. The department must keep **at least two trucks active in the field at all times**: if returning would drop the count below two, the truck stays in the field.

Part 4: Fair refills with Lamport clocks (Estimated duration: ∼6 h)

Goal of Part 4. Apply Lamport logical clocks so that the fire department grants the reservoir in a shared and fair "who asked first" order.

The ordering problem in communication. A server that serves requests in the order they happen to arrive on a socket is at the mercy of OS scheduling and network buffering. That is not a notion of fairness a distributed system should rely on.

Your refill channel now runs over a message broker, but the broker alone does not give you ordering for free. The fire department still serves refill requests in whatever order they happen to reach it. In Task 4 you make refill access **provably fair** using **Lamport logical clocks**.

Fair ordering with Lamport clocks

You must implement a fairness mechanism. Add a Lamport clock to every truck and to the fire-department process. Implement the update rule: increment on every local event; on receive, set $C \leftarrow \max(C, C_{\text{msg}}) + 1$; piggy-back C on every outgoing message. Attach a Lamport timestamp to every refill request. The fire department now accepts incoming requests into a **priority queue** ordered by (*timestamp*, *truck_id*) (the truck ID from your naming scheme breaks ties), and grants the reservoir to the head of the queue.

Tasks in Part 4

a) **(Lamport)** Add a Lamport clock to every truck and to the fire department, with the standard update rule, piggy-backed on every message.

b) **(Fairness)** Have the fire department queue incoming requests in a priority queue ordered by (*timestamp*, *truck_id*) and grant the reservoir to the head of the queue.

c) **(Logging)** Log every truck action (`move`, `refill_request`, `refill_granted`, `extinguish`) with the truck's current Lamport value to a per-truck log file or std-out prefix. The combined log must be sufficient to reconstruct a globally consistent order of events.

Part 5: Prepare a presentation (Estimated duration: ∼6 h)

Goal of Part 5. Communicate your design choices, the resulting architecture, and what you learned, clearly enough that a fellow student could reimplement your system from your slides.

Tasks in Part 5

Prepare a **15-minute presentation** and a **5-minute live demo**. The presentation should cover:

a) **Development process and documentation.** How did you organise the work as a team (Kanban, issue tracking)? How did you review code? How did you make design decisions?

b) **The technical solution.** What application-layer protocol did you design for refills, and why? Which naming scheme did you choose? How did you implement fair mutual exclusion on the water reservoir? How does your call-for-help protocol work?

c) **The architecture.** Draw your final architecture (processes, communication channels, the NATS broker). Identify the architectural style. Discuss *distribution transparency* and *scalability*: where would your design start to hurt if you scaled from 2 to 200 trucks?

d) **Two highlights.** Pick two aspects of your solution that you are especially proud of.

Evaluation criteria

You are allowed, and encouraged, to use LLM-based tools (ChatGPT, Copilot, Claude, ...) as long as you can defend every line of code you submit. To pass the lab you must be able to demonstrate and explain:

- that you applied a suitable development process;
- the application-layer protocol you designed for refills, both at the protocol level (Task 1 wire format) and at the TCP segment level as observed in Wireshark (Task 2);
- how you migrated the refill channel to NATS request–reply, how the trucks coordinate *decentrally* over NATS pub/sub to fight big fires, and the naming scheme that supports this (Task 3);
- how your system ensures fair access to the shared water reservoir with Lamport clocks (Task 4).